How TO Handle Missing Values (using numpy)

---

1. Detect Null Values

Using Pandas

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Age': [25, np.nan, 30, 40],
    'Name': ['John', None, 'Alice', 'Bob'],
    'Salary': [50000, 60000, np.nan, 70000]
})

print(df)
```

Output

```
    Age   Name   Salary
0  25.0   John  50000.0
1   NaN   None  60000.0
2  30.0  Alice      NaN
3  40.0    Bob  70000.0
```

Check null values

```
df.isnull()
```

or

```
df.isna()
```

Count null values

```
df.isnull().sum()
```

---

2. Replace All Null Values with One Value

Replace every null value with `0`

```
df.fillna(0)
```

Output

```
    Age   Name Salary
0 25.0   John 50000
1 0      0    60000
2 30.0 Alice 0
3 40.0 Bob 70000
```

Replace with a string

```
df.fillna("Unknown")
```

---

3. Replace Null Values in a Specific Column

Numeric column

```
df["Age"] = df["Age"].fillna(18)
```

String column

```
df["Name"] = df["Name"].fillna("Not Available")
```

---

4. Replace Using Mean

Best for numerical columns.

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

Example

```
Age values

25
NaN
30
40

Mean = (25+30+40)/3 = 31.67
```

Result

```
25
31.67
30
40
```

---

5. Replace Using Median

Useful when there are outliers.

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Example

```
10
15
20
500
NaN
```

Median = 17.5

---

6. Replace Using Mode

Mostly used for categorical columns.

```
df["Name"] = df["Name"].fillna(df["Name"].mode()[0])
```

Example

```
Apple
Mango
Apple
NaN
```

Result

```
Apple
Mango
Apple
Apple
```

---

7. Forward Fill (ffill)

Copies the previous value.

```
df.fillna(method="ffill")
```

or

```
df.ffill()
```

Example

Before

```
10
NaN
NaN
20
```

After

```
10
10
10
20
```

---

8. Backward Fill (bfill)

Copies the next value.

```
df.fillna(method="bfill")
```

or

```
df.bfill()
```

Example

Before

```
10
NaN
NaN
20
```

After

```
10
20
20
20
```

---

9. Replace Using Dictionary

Different values for different columns.

```
df.fillna({
    "Age": 18,
    "Name": "Unknown",
    "Salary": 50000
})
```

---

10. Replace Null Values Using `replace()`

Replace only `np.nan`

```
df.replace(np.nan, 0)
```

Replace `None`

```
df.replace(None, "Unknown")
```

---

11. Interpolation

Used for continuous numerical data.

```
df["Age"] = df["Age"].interpolate()
```

Example

Before

```
10
NaN
30
```

After

```
10
20
30
```

---

12. Drop Instead of Fill

Remove rows containing null values.

```
df.dropna()
```

Remove columns

```
df.dropna(axis=1)
```

---

13. Fill Only Numeric Columns

```
numeric_cols = df.select_dtypes(include=np.number).columns

df[numeric_cols] = df[numeric_cols].fillna(0)
```

---

14. Fill Only Object (String) Columns

```
object_cols = df.select_dtypes(include="object").columns

df[object_cols] = df[object_cols].fillna("Unknown")
```

---

15. Fill Nullable Integer Columns

Pandas supports nullable integer types (`Int64`).

```
df["Marks"] = df["Marks"].astype("Int64")

df["Marks"] = df["Marks"].fillna(0)
```

---

16. Fill Boolean Columns

```
df["Passed"] = df["Passed"].fillna(False)
```

---

17. Fill Datetime Columns

```
df["JoiningDate"] = df["JoiningDate"].fillna(pd.Timestamp("2025-01-01"))
```

Or use the current date and time:

```
df["JoiningDate"] = df["JoiningDate"].fillna(pd.Timestamp.now())
```

---

18. Fill with Previous Group Value

Useful in grouped data.

```
df["Salary"] = (
    df.groupby("Department")["Salary"]
      .transform(lambda x: x.fillna(x.mean()))
)
```

Each department gets its own mean.

---

19. Using NumPy

Create array

```
import numpy as np

arr = np.array([10, np.nan, 20, np.nan, 30])
```

Replace NaN with 0

```
arr = np.nan_to_num(arr, nan=0)
```

Output

```
[10. 0. 20. 0. 30.]
```

Replace with another value

```
arr = np.nan_to_num(arr, nan=99)
```

Output

```
[10. 99. 20. 99. 30.]
```

---

20. Replace Infinity Too

```
arr = np.array([1, np.nan, np.inf, -np.inf])

arr = np.nan_to_num(
    arr,
    nan=0,
    posinf=999,
    neginf=-999
)
```

Output

```
[1. 0. 999. -999.]
```

---

21. Fill Missing Values Conditionally

Replace missing salaries only where age is greater than 30.

```
df.loc[df["Age"] > 30, "Salary"] = (
    df.loc[df["Age"] > 30, "Salary"].fillna(50000)
)
```

---

22. Replace Nulls In-place

Modify the original DataFrame directly.

```
df.fillna(0, inplace=True)
```

Or for a single column:

```
df["Age"].fillna(18, inplace=True)
```

Note: Chained `inplace=True` operations on a column (like `df["Age"].fillna(..., inplace=True)`) are discouraged in recent Pandas versions. A safer pattern is:

```
df["Age"] = df["Age"].fillna(18)
```

Common Strategies by Data Type

Data Type	Recommended Fill Method	Example
Integer	0, median, mean	`df["Age"] = df["Age"].fillna(df["Age"].median())`
Float	Mean, median, interpolation	`df["Price"] = df["Price"].fillna(df["Price"].mean())`
String/Object	`"Unknown"`, mode	`df["City"] = df["City"].fillna("Unknown")`
Category	Mode or `"Other"`	`df["Color"] = df["Color"].fillna(df["Color"].mode()[0])`
Boolean	`True` or `False`	`df["Active"] = df["Active"].fillna(False)`
Datetime	Fixed date, current timestamp, forward/backward fill	`df["Date"] = df["Date"].fillna(pd.Timestamp.now())`
Time Series	Forward fill, backward fill, interpolation	`df["Value"] = df["Value"].interpolate()`

Summary of Useful Functions

Function	Purpose
`isnull()` / `isna()`	Detect missing values
`notnull()` / `notna()`	Detect non-missing values
`fillna()`	Fill missing values
`replace()`	Replace specific values (including `NaN`)
`ffill()`	Forward fill from previous value
`bfill()`	Backward fill from next value
`interpolate()`	Estimate missing numeric values
`dropna()`	Remove rows or columns with missing values
`np.nan_to_num()`	Replace `NaN` and infinities in NumPy arrays